

10 分钟 Oral 讲稿

共 42 张幻灯片，按页标记 [P1]–[P42]。约 2500 字，正常语速 10 分钟。

每个 [...] 为翻页提示，可自行调整节奏。

开场（约 30 秒）

[P1 — 标题页]

各位老师好，我是张健宁，来自中山大学。我今天报告的题目是《基于 PaddleOCR-VL 的电路原理图 OCR 研究与实现》。这是一个关于把多模态大模型用在工程图纸 OCR 上的工作，从数据集构建、模型训练到多维度评估都做了一遍，代码、数据、权重全部开源。

[P2 — 目录]

报告会按这个结构展开：先讲问题背景和数据集构建，然后介绍实验过程和结果，最后做总结。

一、引言（约 1 分钟）

[P3 — 项目背景与动机]

先说一下为什么要做这个事。电路原理图是硬件工程师的日常语言——上面有元件标号、参数值、引脚号、网络标签，全部都是文字。但问题是，现有的 OCR 引擎，包括 PaddleOCR、Tesseract、EasyOCR，没一个能处理电路图。我们试了一下，基座的视觉语言模型——PaddleOCR-VL-0.9B——直接扔一张电路图进去，输出的是 "Service Name / Data Source" 这种预训练幻觉文本，CompF1 等于零，一行都识别不出来。

更麻烦的是，在小数据集上微调 VLM 会出现模态塌缩——模型不看图片，直接背训练集里的高频 token，输出变成 "1, 2, 3, 4..." 这种数字序列。

本项目做的事情就是四件：第一，从头构建一个电路原理图 OCR 标注数据集，经过 7 轮验证；第二，提出用合成文字数据来防止模态塌缩；第三，做了 Phase 1 到 Phase 4 一共 8 组系统消融实验；第四，全开源。

二、数据集构建（约 1.5 分钟）

[P4 — 数据集概览]

数据集方面，训练集 1520 张，测试集和验证集各 150 张，总共标注了 4810 个 OCR 实例。数据来源包括 KiCad 原理图、合成文字图片和真实手机拍照。标注经过了 7 轮递进式验证管线，质量报告里有 12 组可视化对比。

[P5 — 数据规模与来源]

具体构成：1200 张 KiCad 原理图是我自己在 KiCad 里面画的，然后从 SVG 导出为 PNG，GT 直接从 SVG 的 stroked-text 文本元素提取，所以是百分之百对齐的，没有任何 AI 标注。300 张合成文字图片是用 PIL 生成的，模拟 BOM 表格、元件参数表这些结构化文本格式。还有 20 张真实手机拍照，是把原理图打印成 A4 纸然后用手机拍的。测试集 150 张全部是真实 KiCad 电路图，不含任何合成数据。

[P6 — 标注质量保证]

标注质量这边，7 轮验证管线：从 JSON Schema 自动校验、图片路径验证、控制字符扫描，到元件标号正则匹配、参数值单位标准化，再到 KiCad 网表交叉验证，最后人工抽检 10%。质量报告里有 12 组 GT 和图片的可视化对比，确保标注准确。

[P7 — 数据多样性]

数据多样性方面，三种来源覆盖了矢量导出、程序生成和真实拍照三种场景。但也要诚实地说局限性：真实拍照只有 20 张，只占 1.3%，量不够；原理图全部是 KiCad 单一来源，缺少 Eagle、Altium 这些其他 EDA 工具的导出；也没有扫描文档或者模糊、遮挡的样本。

[P8 — 数据集关键指标一览]

这是一张数据集的 infographic 总览图，把核心指标放在一页上，方便快速了解。

[P9 — 元件类型分布]

元件类型分布：一共 10 种元件类型——电阻、电容、电感、二极管、LED、三极管、IC、连接器、晶振、保险丝，4,810 个 OCR 实例。IC 类的占比最高，因为 IC 的引脚表标注行数多。

三、场景稀缺性（约 1.5 分钟）

[P10 — 研究稀缺性]

这个方向到底有没有人做过？学术界，没有专门针对电路原理图 OCR 的公开数据集；现有相关数据集都没有 OCR 标注；主流 VLM-OCR 论文也都没有涉及工程图纸。工业界呢？PaddleOCR、Tesseract、EasyOCR 三款引擎全部失效。PaddleOCR 的基座 NED 是 0.94，说明字符级偏差已经很大了，但是因为它输出的是自由文本，你拿到一串字符串之后你根本不知道哪个标号对应哪个参数值。

[P11 — 工业需求]

工业需求方面，PCB 逆向工程每年的外包市场超过 5 亿美元，其中大约 30% 的时间消耗在原理图重建上——说白了就是把电路图上的文字一个一个手动敲进电脑。人工转录的错误率在 5% 到 15%。全球半导体市场 6000 亿美元，EDA 工具 400 亿美元，BOM 提取是硬件工程师的日常痛点。所以这个事情如果真的做成了，是有实际价值的。

[P12 — 场景独特性]

电路 OCR 和通用 OCR 有什么本质不同？六点：输出结构——通用 OCR 是自由文本，电路 OCR 是结构化的标号加参数值加引脚；文字密度——电路图上一页可能有 30 多个 OCR 实例，通用文档远没这么密；文字方向——通用 OCR 以水平为主，电路图水平、垂直、旋转都有；符号混合——电路图上密集的电气符号和文字交错；领域知识——需要理解 Ω 、F、H、V 这些电子工程单位；评估方式——通用 OCR 看字符级 NED，我们定义了元件级的 CompF1 加 LineAcc。

[P13 — 真实电路图示例]

这就是一张真实的 KiCad 开源项目原理图——Nifty Numpad，基于 RP2040 的键盘。可以看到：高密度的元件排布、多方向的文字、电气符号和标注紧密交错。这就是我们要处理的东西。

四、任务复杂度（约 1 分钟）

[P14 — 视觉复杂度]

我们分析了 7 个维度的视觉复杂度：密文字、多方向、微字体、符号混排、线条穿越等等。在每一个维度上，电路 OCR 的难度都远超通用文档。这张图给了直观对比。

[P15 — 结构复杂度]

更关键的是结构复杂度。电路 OCR 不是简单的文字识别，它隐含了五个子任务：元件检测、参数值读取、标号-值配对、引脚解析、网络标号识别。这五个子任务共享同一个 VLM 解码器，没有显式的任务边界，模型需要隐式地学会多任务协同。其中最核心的指标就是 CompF1——元件级 F1，它衡量的是模型能不能把一个元件的标号和它的参数值正确配对。

五、训练数据集构建（约 0.5 分钟）

[P16 — 数据采集与标注流程]

训练数据这边再强调一下：所有数据版权清晰，MIT 协议。KiCad 原理图是自己画的，合成文字是程序生成的，真实拍照是作者自己拍的。7 轮质量控制每一轮都有明确的通过标准，测试集 150 张全部是真实 KiCad 电路图，评测的时候不掺任何合成数据。

六、实验与结果（约 1.5 分钟）

[P17 — 基座模型 vs 微调模型]

实验这边，先看最核心的结果。基座模型 PaddleOCR-VL-0.9B：CompF1 = 0，LineAcc = 0，完全没法用。exp6，也就是 v1，用 300 张合成文字混合训练之后：CompF1 从 0 升到 0.119，LineAcc 到 0.033——输出开始包含电路领域词汇了。Phase 1，也就是 v2，在 5000 张合成 KiCad 原理图上预训练之后：CompF1 达到 0.304，比 v1 又提升了 2.6 倍，LineAcc 到 0.040。这是我们目前的最优模型。

[P18 — Phase 1 四组控制变量实验]

Phase 1 做了四组控制变量实验。全部四组都出现了模态塌缩——模型不看图片，直接输出 "1, 2, 3, 4..." 这种数字序列。关键发现是：解冻 Projector 是塌缩的根因。Projector 是视觉特征到语言空间的唯一映射桥梁，LoRA 扰动这些权重之后，视觉特征被扭曲了，LLM 看不懂，就退化成了机械重复训练集里的高频 token。所以冻结 Projector，只微调 LLM 的 self-attention 层，是最安全的策略。

[P19 — Phase 2 合成数据防塌缩]

Phase 2 引入合成文字数据来防塌缩。方法是生成 300 张合成文字图片，按 20% 的比例混入训练集。结果：exp5 防过拟合配置的 CompF1 是 0.126，exp6 基线配置是 0.156。相比 Phase 1 里没有合成数据的实验（最高的也就 0.092），CompF1 提升了 4.2 倍。合成数据混合是有效的。

[P20 — 合成数据示例]

这就是合成文字数据的实际样子——模拟 BOM 表格风格，白底黑字，多字号。它的作用是强制模型必须真正读取图片，因为这种随机生成的文字根本没法通过统计模式去"猜"。唯一的降低 loss 的路径就是老老实实看图片。

[P21 — Phase 3/4]

Phase 3 和 Phase 4 尝试了更大的 LoRA 秩和 SPICE 格式微调。结果都不理想：r=32 的 CompF1 只有 0.084，SPICE 微调只有 0.042。说明在目前 1200 样本的数据规模下，增大模型容量和改变输出格式都没有用——根本瓶颈是数据量。

[P22 — 消融总结]

一张消融总结表：六个维度——学习率、分辨率、Dropout、Projector 冻结还是解冻、有没有合成数据、LoRA 秩。核心结论就是三条：合成数据有效，冻结 Projector 必须做，更大的秩在当前数据量下无效。

七、合成 KiCad 预训练（约 1 分钟）

[P23 — 规模化数据生成]

Phase 1 的关键突破是规模化合成 KiCad 数据。我们用 Python 脚本随机组合 10 类元件符号，调用 kicad-cli 命令行工具自动导出 SVG 再转 PNG，生成了 5000 张合成原理图。GT 从 SVG 的 stroked-text 文本元素提取，百分之百对齐。下面这张就是我们实际生成的合成原理图——元件符号、连线、文字标注和真实 KiCad 图纸外观完全一致。

[P24 — 多模型对比]

多模型对比这张图非常直观：基座模型 CompF1 为零，v1 有了一点但很低，v2 显著拉升。

[P25 — 消融实验汇总]

消融实验汇总图：展示了两组核心对比——合成预训练的效果和两阶段训练的效果。

[P26 — 合成预训练实验结果]

数值总结：v1 CompF1 = 0.119，v2 CompF1 = 0.304，提升 2.6 倍；LineAcc 从 0.033 到 0.040，提升 1.2 倍。但也要注意：JointF1-norm 双方都接近零，标号-参数值的联合识别仍然是核心瓶颈。而且 v2 加上真实数据两阶段微调之后性能反而退化到了 0.002——说明从合成到真实的领域迁移不是简单的 fine-tune 能解决的。

八、讨论（约 0.5 分钟）

[P27 — 合成数据为何有效]

合成数据为什么能防塌缩？机制分析：合成图片具有确定的视觉到文字的映射关系，模型没办法通过统计模式去猜，唯一的降低 loss 路径就是真正读取图片，这样就强制保持了视觉编码器的活跃。20% 是最优混合比例——太低了防塌缩效果不足，太高了会过拟合简单的白底黑字视觉风格。这个方法具有通用性，任何小数据集上的 VLM-OCR 任务都可以复用。

[P28 — 当前瓶颈与局限]

当前的主要瓶颈：数据方面，1200 样本不够；CompF1 虽然到了 0.304，但 LineAcc 只有 0.040，行级匹配率还很低；更大秩在当前数据量下无效。模型方面，复杂样本像 ESP32 的引脚表仍然持续塌缩；参数值幻觉没有解决——模型有时候会自己编造电阻值和电容值；ExactMatch 是零——没有任何一张测试图片被完整正确识别。0.9B 的参数容量也可能是一个限制。

九、可投入应用程度分析（约 0.5 分钟）

[P29 — 最优模型能力画像]

看一下 v2 模型目前真实的能力水平。已建立的能力：电路图的视觉风格表征、不塌缩的稳定输出、大约 30% 的元件标号检出率。未建立的能力：标号和参数值的配对、复杂引脚表的识别、结构化的输出格式。当前阶段是 TRL 3 概念验证，部分能力接近 TRL 4。

十、识别案例展示（约 1 分钟）

接下来展示 5 个真实案例的三模型对比——基座、exp6、Phase 1——让大家直观感受模型的演进。

[P30–P31 — 案例 0]

案例 0，1446.png，17 个元件。基座输出的是 "Service Name / Status / Version" 这种 API 文档模板；exp6 塌缩成了 "1, 2, 3..." 数字序列；Phase 1 的前 21 行和 GT 完全一致——H101 到 H105 的标号和 6.00mm 的尺寸标注、+5V 和 +3.3V 的电压标签、SWD Connector、USB-C 等全部正确。

[P32–P33 — 案例 1]

案例 1，0385.png，22 个元件。基座输出 "This image is a graphic design..." 图像描述；exp6 输出 "Model / T-SNE / PCA / Data / Analysis..." 机器学习术语；Phase 1 的前 20 行和 GT 完全一致——PWR_FLAG、+12V_In、DC-005-5A-2.0 这些全部正确。

[P34–P35 — 案例 5]

案例 5，0478.png，36 个元件。基座是填表模板，exp6 是 +3.3V 重复 19 行——这很有意思，它其实识别到了电源网络的存在，但是产生了数值幻觉，就一直重复。Phase 1 有差异化输出——+3.3V、GND、+5V、VBUS、D+、D-、CC1、SBU1、R1/5.1k、U1/ESP32 等等，说明模型对不同区域产生了不同的响应。

[P36–P37 — 案例 8]

案例 8，0640.png，42 个元件。基座是数据表格描述，exp6 把 AMP POWER 误识成了 APP P7016 然后塌缩为数字序列。Phase 1 的前 13 行和 GT 完全一致——AMP POWER 正确恢复，1/2/3 引脚号、+12V、GND×5、OUTPUT×2 全部正确。

[P38–P39 — 案例 12]

案例 12，1449.png，29 个元件。基座是图像说明文本，exp6 和案例 5 一样是 +3.3V 幻觉。Phase 1 输出了多种电压等级和元件内容——+3.3V、GND、+5V、+12V、R1/10k、R2/22k、R3/47k、C1/100nF、C2/10uF、U1/LM358、D1/1N4148——对复杂电路的不同区域产生了差异化输出。

十一、总结（约 0.5 分钟）

[P40 — 核心发现]

总结一下核心发现。有效的手段：合成数据混合，CompF1 提升 4.2 倍；防过拟合配置——低学习率加高 dropout；冻结 Projector 加 LoRA 的训练策略；合成 KiCad 预训练——CompF1 达到 0.304，2.6 倍提升。无效的手段：512px 高分辨率、解冻 Projector、更大的 LoRA 秩、SPICE 格式微调、以及合成到真实的两阶段微调——这些在当前数据规模下都没有带来增益。

[P41 — 开源资产]

所有资产全部开源，MIT 协议：代码仓库、数据集仓库、HuggingFace 在线 Demo、LoRA 权重、中英文技术报告、以及本场演示的 Beamer 文件——全部可以公开获取。

[P42 — 谢谢]

谢谢大家，欢迎提问。

时间分配参考： 开场 0.5' | 引言 1' | 数据集 1.5' | 场景稀缺性 1.5' | 任务复杂度 1' | 数据集构建 0.5' | 实验 1.5' | KiCad预训练 1' | 讨论 0.5' | 可部署性 0.5' | 案例 1' | 总结 0.5'